

UJWAL NAGRIKAR

+91-84463-62075 | ujjwalnagrikar@gmail.com | [linkedin.com/in/ujwal-nagrikar](https://www.linkedin.com/in/ujwal-nagrikar) | github.com/UjwalNagrikar

SUMMARY

DevOps Engineer focused on cloud-native platforms, Kubernetes, and infrastructure automation across AWS and GCP. Experienced in building and managing containerized environments with Terraform, Helm, GitOps, Jenkins, GitHub Actions, Prometheus, and Grafana. Apply SRE practices such as SLIs/SLOs, monitoring, incident response, automated remediation, capacity planning, and continuous improvement to maintain dependable systems. Hands-on experience with GPU-based AI infrastructure and LLM deployment using vLLM, NVIDIA CUDA, and Tesla T4. Proficient in Python, Bash, networking, and cloud technologies, with an emphasis on secure deployments, efficient operations, troubleshooting, and adapting to emerging technologies.

SKILLS

- **Platforms & Tools:** AWS, GCP, Docker, Kubernetes, Terraform, Ansible, Helm, ArgoCD, Jenkins, GitHub Actions, GitLab CI/CD, Linux
- **Monitoring & Observability:** Prometheus, Grafana, ELK Stack
- **Programming & Scripting:** Python (Automation, Boto3, Cloud SDKs), Bash, SQL
- **DevOps Practices:** CI/CD, Infrastructure as Code (IaC), GitOps, Infrastructure Automation, Configuration Management, Containerization
- **AI/LLM & GPU Infrastructure:** LLM Deployment, Model Serving, Inference Infrastructure, GPU Workloads, GPU Orchestration, NVIDIA GPUs, CUDA, Kubernetes GPU Scheduling, NVIDIA GPU Operator, GPU Resource Management

PROJECTS

Kubernetes Observability & Automated Remediation [GitHub](#)

Kubernetes · Prometheus · Grafana · Alertmanager · Python · Bash · ArgoCD · Helm

- Defined an availability SLO of 99.9% backed by SLIs for request success rate, request latency (p95/p99), error rate, and pod availability using error budgets to drive release and reliability decisions.
- Instrumented workloads with custom Prometheus metrics and built Grafana dashboards tracking the golden signals (latency, traffic, errors, saturation) for real-time service health visibility.
- Configured Alertmanager rules for high error rate, high latency, pod crash-loops, node CPU exhaustion, memory pressure, and failed deployments, routing targeted, actionable alerts and reducing alert noise.
- Built a closed-loop automated remediation pipeline (Prometheus detects → Alertmanager → Python/Bash automation restart/rollback → service restored) that auto-recovers common failures and reduces MTTR.
- Automated health checks and post-deploy smoke tests with ArgoCD drift detection and automated rollback and tracked availability, MTTR, error rate, deployment frequency, and change-failure rate to quantify stability.

Automated CI/CD Platform with Security Enforcement on GKE [GitHub](#)

GCP GKE · Terraform · GitHub Actions · ArgoCD · Helm · Prometheus · Grafana · Loki · Trivy

- Provisioned GKE infrastructure with Terraform (VPC, private cluster, node pools, IAM, remote state), reducing environment provisioning from 2 hours to under 20 minutes (83% faster) with fully repeatable, version-controlled builds.
- Built a GitOps CI/CD pipeline with GitHub Actions, ArgoCD, and Helm that automates build, security-scan, image-promotion, and deployment across dev and staging, eliminating all manual release steps.
- Integrated Trivy container image scanning and vulnerability gates into the CI pipeline to detect and block high/critical vulnerabilities before artifacts are promoted to deployment environments.
- Implemented Prometheus, Grafana, Loki, and Alertmanager for centralized metrics, logs, dashboards, and actionable alerts across Kubernetes workloads, improving visibility into application and cluster health.
- Defined SLOs and configured Alertmanager rules to fire targeted alerts when platform behavior drifted from expected thresholds, replacing manual log-watching and speeding incident response.
- Managed 15 Kubernetes workloads with ArgoCD drift detection and automated sync, reducing manual deployment effort by 80% and tracking delivery via DORA metrics.

Production-Grade vLLM Deployment on Kubernetes [GitHub](#)

AWS · K3s · Kubernetes · vLLM · Qwen2.5 · NVIDIA T4 · CUDA · Helm · Terraform · Prometheus · Grafana · Go · Linux

- Engineered a GPU-powered LLM inference platform on AWS using K3s, vLLM, and Qwen2.5-3B-Instruct, integrating the NVIDIA Container Toolkit and Kubernetes GPU device plugin to expose and schedule 1x Tesla T4 (15 GB VRAM) for model serving.
- Diagnosed and resolved GPU-runtime and inference compatibility issues across CUDA, NVIDIA runtime, containerd, and vLLM, including a Tesla T4 sampling compatibility issue, restoring stable inference with 0 pod restarts during validation.
- Automated infrastructure and application deployment using Terraform + Helm, provisioning the cloud environment and deploying version-controlled Kubernetes workloads with resource requests/limits, persistent model storage, health checks, and GPU-aware scheduling.
- Implemented observability with Prometheus + Grafana, monitoring Kubernetes and inference infrastructure including GPU utilization/memory, CPU, memory, pod health, request metrics, and latency, providing centralized visibility for operational troubleshooting.
- Exposed and validated an OpenAI-compatible inference API, achieving HTTP 200 responses for /v1/models and /v1/chat/completions with Qwen2.5-3B-Instruct, enabling application integration while validating the complete GPU → Kubernetes → vLLM → API → monitoring pipeline.

EDUCATION

G. H. Raisoni College of Engineering & Management, Nagpur, India

Bachelor of Commerce in Computer Applications (BCCA)

2024 – 2027